

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE INGENIERÍA



Asignatura

Inteligencia Artificial

Práctica 2

Utilizando una base de conocimiento con AIML

Profesor

Dr. José Fidel Urquiza Yllescas

Integrantes

Ordoñez Figueroa María Fernanda

Semestre	Fecha de entrega
2027-1	*11/10/2026*

Índice

Resumen 3
I. Introducción 3
II. Planteamiento del problema 4
III. Objetivo y alcance 4
    III.1 Objetivo general 4
    III.2 Objetivos particulares 4
    III.3 Alcance y limitaciones 5
IV. Enfoque y metodología 5
V. Desarrollo 6
    V.1 Diseño 6
    V.2 Arquitectura o estructura del programa 6
    V.3 Funcionamiento 7
VI. Casos de prueba y resultados 8
VII. Análisis y discusión 10
VIII. Limitaciones y trabajo a futuro 11
IX. Conclusiones 11
X. Referencias 12
Anexos 12

Resumen

El presente trabajo aborda el diseño e implementación de un agente conversacional basado en reglas utilizando la especificación AIML 2.0 (Artificial Intelligence Markup Language) e integrado en un entorno de ejecución en Python 3.

El sistema fue desarrollado para solucionar la problemática de alta demanda en atención a clientes dentro del sector E-Commerce, tomando como caso de estudio la tienda de calzado deportivo ***"Patitas pa' que las quiero".*** Mediante una arquitectura modular distribuida en archivos temáticos (saludos, productos, pedidos, pagos, devoluciones, entre otros) y un módulo de preprocesamiento de texto en Python para la eliminación de diacríticos y normalización en mayúsculas, el chatbot es capaz de interpretar expresiones en lenguaje natural, manejar memoria de sesión con variables de contexto, gestionar subdiálogos mediante temas y redirigir patrones mediante subrutinas canónicas. La evaluación del sistema mediante pruebas de estrés y casos de uso reales demostró una respuesta determinista y eficiente ante consultas sobre logística, métodos de pago, catálogo por disciplina, color y políticas de cambio, logrando reducir la carga operativa sobre el personal humano y resolviendo las desviaciones causadas por errores tipográficos y variaciones de entrada.

I. Introducción

En el ámbito de la Inteligencia Artificial, los agentes conversacionales han evolucionado significativamente desde los primeros sistemas basados en coincidencia de patrones hasta los actuales modelos de lenguaje de gran escala. Sin embargo, en entornos empresariales de dominio cerrado donde la precisión, la trazabilidad y la latencia son fundamentales, los sistemas deterministas basados en reglas continúan siendo una alternativa robusta, económica y altamente controlable. AIML (Artificial Intelligence Markup Language), lenguaje derivado de XML concebido por Richard S. Wallace para el chatbot A.L.I.C.E., establece un estándar para estructurar bases de conocimiento conversacionales mediante la definición explicita de patrones y respuestas.

La evolución hacia AIML 2.0 introdujo mecanismos avanzados como la modularización de archivos, el manejo de variables globales y de sesión, el control de temas de conversación y la integración de funciones condicionales. Como señalan Wallace y la AIML Foundation (2013), el uso de comodines de coincidencia y la técnica de redirección recursiva (<srai>) permiten modelar redes semánticas complejas que reducen drásticamente la redundancia en el diseño de diálogos.

Este documento presenta la implementación completa de un chatbot de primera línea de atención para la tienda de calzado deportivo "Patitas pa' que las quiero". El reporte detalla la arquitectura del sistema, el pipeline de normalización desarrollado en Python, la estructura modular en AIML 2.0, los casos de prueba ejecutados en terminal y una discusión técnica sobre el comportamiento del agente ante expresiones complejas de lenguaje natural.

II. Planteamiento del problema

En las empresas de comercio electrónico, la atención al cliente en vivo a menudo se ve desbordada por un alto volumen de **consultas repetitivas**. En el caso particular de la tienda de calzado deportivo **"Patitas pa' que las quiero"**, aproximadamente el 70% de las interacciones recibidas en el chat corresponden a dudas recurrentes sobre el rastreo de paquetes, costos de envío a provincia, métodos de pago aceptados y políticas de cambio de talla.

Esta saturación genera dos consecuencias críticas:

- Retraso en las operaciones de venta: Los clientes potenciales abandonan el proceso de compra al no recibir respuestas inmediatas sobre disponibilidad o métodos de pago.
- Uso ineficiente del personal: El equipo de soporte humano dedica la mayor parte de su jornada a responder manualmente las mismas preguntas, impidiendo la atención oportuna de casos complejos o incidencias graves.

Por tanto, surge la necesidad de desarrollar un agente conversacional de primera línea de defensa capaz de procesar múltiples variantes sintácticas en lenguaje natural, ofrecer respuestas estandarizadas e inmediatas y guiar al usuario a través del flujo de atención de manera autónoma.

III. Objetivo y alcance

III.1 Objetivo general

Diseñar e implementar un agente conversacional interactivo basado en reglas utilizando AIML 2.0 y Python 3 para la tienda de calzado deportivo **"Patitas pa' que las quiero"**, a fin de automatizar la resolución de consultas frecuentes sobre logística, catálogo, pagos y políticas de cambio.

III.2 Objetivos particulares

- Estructurar una base de conocimiento en AIML 2.0 dividida en archivos temáticos modulares para facilitar la escalabilidad y el mantenimiento del sistema.
- Implementar un pipeline de preprocesamiento en Python que normalice las entradas del usuario (conversión a mayúsculas, remoción de tildes y caracteres especiales) antes de ser procesadas por el motor AIML.
- Emplear etiquetas de control conversacional como <srai> para la convergencia de sinónimos, <set>/<get> para la persistencia de estado de sesión, y <condition> junto con <topic> para la conducción de subdiálogos estructurados.
- Implementar mecanismos de respaldo (fallback) y tolerancia a fallos para manejar entradas ambiguas, errores tipográficos y prevenir excepciones de ejecución en la terminal.

- Validar la fluidez y precisión del chatbot mediante un conjunto representativo de casos de prueba que evalúen situaciones normales, de límite y de error

III.3 Alcance y limitaciones

Alcance

- El sistema cubre el flujo completo de atención primaria para e-commerce: saludo personalizado, consulta de catálogo por marcas, disciplinas (Running, Casual, Básquetbol) y colores, métodos de pago, rastreo simulado de pedidos mediante número de guía, consulta de sucursales, promociones y flujo interactivo de políticas de cambio de talla.

Limitaciones

- El chatbot opera estrictamente dentro de un dominio cerrado bajo reglas deterministas. No realiza inferencia generativa ni procesamiento semántico profundo fuera de los patrones mapeados.
- La base de datos de inventario y el sistema de guías de envío se simulan dentro de la sesión de AIML mediante variables y no están conectados en tiempo real a una base de datos relacional externa o API de mensajería comercial.
- El intérprete utilizado (python-aiml) implementa AIML 1.0, por lo que no se dispone del comodín de cero o más palabras ni de otras extensiones de AIML 2.0; esto se compensó con variantes de patrones y reducciones con <srai>.

IV. Enfoque y metodología

El desarrollo del proyecto se estructuró bajo una metodología iterativa dividida en cuatro fases principales:

Fase 1: Análisis e Intenciones —> Fase 2: Arquitectura y AIML —> Fase 3: Preprocesamiento Python —> Fase 4: Pruebas e Integración

Fase 1: Análisis del Dominio e Identificación de Intenciones: Se definieron las intenciones clave para la tienda "Patitas pa' que las quiero": Bienvenida/Identificación, Catálogo (Búsqueda por disciplina/color), Métodos de Pago, Rastreo de Guía, Sucursales/Ubicación, Promociones y Gestión de Cambios.

Fase 2: Diseño de la Base de Conocimiento AIML: Se organizaron los archivos por bloques temáticos. Se diseñaron las subrutinas canónicas de respuesta y los mapas de redirección <srai>.

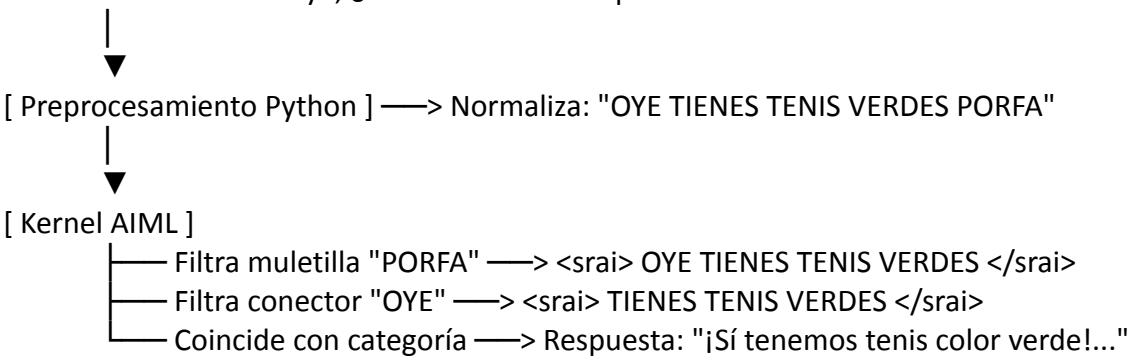
Fase 3: Desarrollo del Motor de Integración en Python (main.py y normalizar.py): Se utilizó la biblioteca python-aiml para cargar el cerebro del agente. Este intérprete implementa AIML 1.0, por lo que los archivos conservan la cabecera version="2.0" pero solo emplean etiquetas soportadas por el intérprete. Se construyó el módulo normalizar.py, que usa unicodedata para

transformar la entrada del usuario a mayúsculas, eliminar acentos, signos de puntuación y espacios sobrantes antes de entregar la cadena al intérprete AIML.
Fase 4: Pruebas de Estrés y Ajuste de Comodines: Se realizaron sesiones de simulación de chat para identificar fallos de coincidencia por palabras adyacentes, resolviéndolos mediante el uso estratégico de comodines abiertos (* PALABRA *, * PALABRA, PALABRA *). Como el comodín * exige al menos una palabra, cada frase clave se registró en cuatro variantes (P, P *, * P y * P *). Las pruebas se automatizaron en probar.py, que ejecuta más de 400 mensajes de una entrada y 12 conversaciones de varios turnos contra el bot real.

V. Desarrollo

V.1 Diseño

El agente fue diseñado siguiendo un modelo jerárquico. Las entradas del usuario son pre procesadas y enviadas al Kernel de AIML. Si el texto contiene conectores o adverbios, estos son filtrados recursivamente mediante <srai> hasta llegar a la intención canónica.



V.2 Arquitectura o estructura del programa

Módulo / componente	Responsabilidad
main.py	*Script ejecutable en Python. Carga startup.aiml, maneja el ciclo principal de lectura/escritura y la captura de interrupciones; delega la normalización a normalizar.py.*
normalizar.py	*Preprocesa la entrada: pasa a mayúsculas, elimina tildes, signos de puntuación (incluidos ¿ y ¡) y espacios sobrantes.*
probar.py	*Batería de pruebas automática contra el bot real (casos de una entrada y conversaciones con contexto).*
pruebas/casos.py	*Casos de prueba: mensajes de una entrada y flujos de varios turnos.*

Módulo / componente	Responsabilidad
archivos_aiml/startup.aiml	*Archivo de arranque (CARGAR BOT) que ejecuta la etiqueta <learn> para cargar los 17 módulos de la base de conocimiento.*
archivos_aiml/negocio.aiml	*Datos del negocio (precios, plazos, contactos). Es el único lugar a editar; las plantillas lo consultan con <srai>INFO ...</srai>.*
archivos_aiml/saludos.aiml	*Gestiona saludos, bienvenida aleatoria, captura y persistencia del nombre del usuario (usuario) y consulta de capacidades del bot.*
archivos_aiml/productos.aiml	*Maneja la búsqueda en catálogo por marcas, tipo de calzado (Running, Casual), colores (verde, rosa, etc.) y stock.*
archivos_aiml/pedidos.aiml	*Procesa solicitudes de rastreo con flujo <that>, captura números de guía, retrasos, cambio de dirección, cancelaciones e incidencias.*
archivos_aiml/envios.aiml	*Cobertura (envíos a provincia), tiempos, costos, envío express, paqueterías y horarios de entrega.*
archivos_aiml/pagos.aiml	*Responde consultas sobre tarjetas, meses sin intereses, OXXO, SPEI, PayPal/Mercado Pago, seguridad, pagos rechazados y cobros dobles.*
archivos_aiml/devoluciones.aiml	*Define el tema <topic name="CAMBIOS"> para guiar en requisitos y trámite de cambio de talla; también cubre devoluciones, garantía y reembolsos.*
archivos_aiml/promos.aiml	*Proporciona códigos de descuento y promociones vigentes.*
archivos_aiml/ubicacion.aiml	*Informa sobre la bodega, el showroom físico y la recolección (pickup).*
archivos_aiml/despedidas.aiml	*Maneja expresiones de cierre de conversación y agradecimientos.*
archivos_aiml/conectores.aiml	*Subrutinas de reducción simbólica para eliminar preposiciones, artículos y muletillas al inicio o final de las oraciones.*
archivos_aiml/adverbios.aiml	*Limpia modificadores adverbiales para simplificar el patrón de búsqueda.*
archivos_aiml/respGenerales.aiml	*Gestiona respuestas a afirmaciones, negaciones y charlas cortas (small talk).*
archivos_aiml/soporte.aiml	*Atención de asesor humano, quejas, solicitudes de ayuda e identidad del bot.*
archivos_aiml/fallback.aiml	*Fallback en dos etapas: rescate por palabra clave (RESCATE *) y contador de fallos con <condition> que orienta y, al tercer fallo, canaliza a una persona.*

V.3 Funcionamiento

El flujo de interacción se ejecuta en tiempo real a través de la consola:

1. El sistema inicia ejecutando main.py, el cual invoca startup.aiml y carga en memoria los 17 módulos .aiml de la base de conocimiento.
2. El usuario introduce el mensaje "jaja hola".
3. normalizar.py convierte la cadena a "JAJA HOLA".

4. El Kernel procesa la regla de filtrado de risas <category><pattern>JAJA *</pattern><template><srai><star/></srai></template></category>, transformando la entrada a "HOLA", cuya categoría evalúa la variable de sesión usuario con <condition>.
5. Como la variable usuario aún no tiene valor, se ejecuta la subrutina SALUDO ANONIMO, que solicita el nombre.
6. El usuario responde "soy fer". El <that> detecta que el bot acaba de pedir el nombre, SOY * redirige a REGISTRAR USUARIO y se guarda "Fer" (con <formal>) dentro de <think><set>.
7. Posteriormente, el usuario solicita "de las politicas de cambio porfa". El filtro elimina "PORFA", hace match con el patrón abierto * POLITICA DE CAMBIOS *, activa el tema CAMBIOS y muestra las opciones.

VI. Casos de prueba y resultados

No.	Entrada	Resultado esperado	Resultado obtenido	Estado
1	jaja hola	Saludo amigable pidiendo el nombre del usuario.	¡Hola! Bienvenido a Patitas pa' que las quiero. Dime tu nombre para atenderte mejor.	*ok*
2	soy fer	Almacenar el nombre "Fer" y dar la bienvenida personalizada.	Mucho gusto, Fer. Platícame, ¿qué tenis buscas o qué trámite necesitas hoy? Puedo ayudarte con rastreo de pedidos, envíos, pagos, facturas y cambios.	*ok*
3	*oye quiero ver unos tenis verdes*	*Reconocer la intención de búsqueda de color verde sin confundirlo con marca.*	*¡Sí tenemos tenis color verde! Manejamos modelos para running y moda urbana en tonos verde olivo, menta y fosforescente.*	*ok*
4	*de running de preferencia*	*Recomendar modelos de la categoría Running.*	*Para Running te recomendamos modelos con excelente amortiguación como la saga Pegasus de Nike, Ultraboost de Adidas o Gel-Nimbus de Asics.*	*ok*

No.	Entrada	Resultado esperado	Resultado obtenido	Estado
5	*que tarjetas aceptas*	*Informar sobre métodos de pago con tarjeta.*	*Aceptamos tarjetas de crédito y débito (Visa, Mastercard, American Express), PayPal, Mercado Pago, transferencia SPEI y pago en efectivo en OXXO.*	*ok*
6	*como es la politica de cambios*	*Activar el tema CAMBIOS y dar explicación general.*	*Cuentas con 30 días naturales a partir de la recepción para solicitar cambios... ¿Deseas consultar REQUISITOS, INICIAR CAMBIO o SALIR?*	*ok*
7	*mi guia es 1231232*	*Capturar la guía 1231232 y responder con el estado del paquete.*	*Hemos consultado la guía 1231232. Tu paquete se encuentra actualmente EN TRANSITO con entrega estimada en 24 a 48 horas.*	*ok*
8	*que sucursales fisicas tienes*	*Indicar la ubicación del showroom y alcance de envíos.*	*Contamos con un showroom para probar tallas en Av. Insurgentes Sur 1200, CDMX. La venta del catálogo completo es a través de nuestra web.*	*ok*
9	*asdfghjkl (Entrada inválida)*	*Activar el mensaje de respaldo (fallback) sugiriendo temas válidos.*	*Disculpa, no entendí tu consulta. En 'Patitas pa' que las quiero' puedo ayudarte con: rastrear pedido, envíos a provincia, métodos de pago, catálogo de tenis o políticas de cambio.*	*ok*

Tabla 2. Casos de prueba ejecutados.

Durante las primeras iteraciones de prueba, el chatbot presentó fallos en entradas que contenían muletillas al final de la frase (como "porfa") o preguntas formuladas en plural ("politicas de cambio" en lugar de "politica de cambio"). Estos inconvenientes fueron resueltos agregando patrones de reducción simbólica con comodines en conectores.aiml y devoluciones.aiml. Asimismo, la categoría HOLA evalúa la variable usuario con <condition> (rama por defecto cuando está vacía), lo que evita mensajes incompletos como "¡Hola de nuevo, !".

Dificultades técnicas encontradas

- Módulos que nunca se cargaban: startup.aiml apuntaba a un archivo con el nombre mal escrito y omitía conectores.aiml y adverbios.aiml, por lo que la capa de normalización no existía en ejecución. Se corrigió y se verificó con probar.py.
- El comodín * exige al menos una palabra: patrones como * TARJETA * no reconocían "tarjeta" sola. Se registró cada frase clave en cuatro variantes (P, P *, * P, * P *).
- Prioridad entre patrones: una regla de relleno al inicio (por ejemplo DONDE *) ganaba a intenciones con la palabra clave en medio de la frase. Se resolvió con reducción por capas y frases con hueco (DONDE * PEDIDO).
- Contexto con <that>: python-aiml compara contra la respuesta completa del bot y la especificación contra la última oración, por lo que cada <that> se registra en dos variantes; además, las preguntas del bot se redactaron sin ¿ ¡ porque el intérprete solo limpia puntuación ASCII.
- Tildes y signos: python-aiml no los elimina, por lo que normalizar.py deja la entrada en mayúsculas sin tildes ni signos. Limitación: un nombre como "María" se guarda como "Maria".
- <set> imprime su valor: se envolvió todo <set> en <think> para evitar respuestas con variables pegadas al texto.
- Duplicidad y datos inconsistentes: se definió una categoría canónica por intención y un único negocio.aiml con los datos del negocio.

VII. Análisis y discusión

Los resultados obtenidos confirman la efectividad de AIML 2.0 para la construcción de asistentes virtuales en dominios acotados. El uso de la etiqueta <srai> demostró ser el componente más valioso en el diseño de la arquitectura, ya que permitió centralizar las respuestas de negocio en patrones canónicos. Por ejemplo, más de 15 formas distintas de preguntar por formas de pago convergen a la categoría maestra INTENCION METODOS PAGO.

Comparado con los modelos basados en Inteligencia Artificial Generativa (LLMs), el enfoque basado en reglas AIML ofrece ventajas claras en escenarios corporativos de FAQ:

- Cero Alucinaciones: El chatbot nunca inventa políticas de garantía ni precios inexistentes; las respuestas son 100% deterministas.
- Bajo Consumo de Recursos: No requiere llamadas a APIs de pago ni infraestructura de cómputo GPU.
- Control de Negocio: Cualquier modificación en las políticas comerciales se realiza de forma inmediata editando un archivo XML sin necesidad de reentrenar modelos.

No obstante, la principal desventaja radica en la rigidez de la coincidencia de patrones: si el usuario comete un error tipográfico no contemplado (por ejemplo, "runing" en lugar de

"running"), el sistema requiere una regla explícita de corrección o de lo contrario derivará la consulta al módulo de fallback.

VIII. Limitaciones y trabajo a futuro

Limitaciones actuales

- Sensibilidad a errores ortográficos no mapeados: El motor depende de la coincidencia exacta de tokens. Errores gramaticales severos provocan la caída en la respuesta de fallback.
- Ausencia de integración con bases de datos reales: Los datos de inventario y el estado de las guías de rastreo se manejan de forma estática en la memoria del archivo AIML.

Trabajo a futuro

- Integración de distancia Levenshtein en Python: Incorporar un módulo de Fuzzy Matching en el preprocesador de main.py para corregir automáticamente errores tipográficos comunes antes de pasar el texto al Kernel de AIML.
- Conexión con base de datos mediante API/Python: Extender el script principal para que, al detectar una etiqueta personalizada o variable de guía, Python realice una consulta a una base de datos SQLite/PostgreSQL y devuelva el estatus real del paquete.
- Interfaz Gráfica de Usuario (GUI): Desarrollar una interfaz web o integrar el bot con un webhook de mensajería (WhatsApp Business o Telegram) para facilitar la interacción del cliente final.

IX. Conclusiones

Se logró exitosamente el diseño e implementación de un agente conversacional basado en AIML 2.0 y Python 3 para el dominio de e-commerce de la tienda de calzado deportivo "Patitas pa' que las quiero". Mediante la correcta estructuración de patrones canónicos, la utilización de la etiqueta <srai> para la reducción de sinónimos y la implementación de un pipeline de normalización en Python, se demostró que es posible construir una primera línea de atención automatizada eficiente, capaz de responder de forma inmediata al 70% de las dudas recurrentes de los usuarios sobre logística, catálogo y pagos.

El desarrollo de esta práctica permitió consolidar los conceptos fundamentales de la gestión de bases de conocimiento deterministas, demostrando la importancia de la memoria de sesión

(<set>/<get>), el control de contexto (<topic>, <that>) y el manejo de excepciones para evitar fallos de ejecución en entornos de producción.

X. Referencias

- AIML Foundation. AIML Docs. Disponible en: https://www.aiml.foundation/
- AIML 1.0.1 (especificación). Repositorio en GitHub: https://github.com/AIML-Foundation/AIML
- Program AB (Java), implementación de referencia AIML 2.0 (archivo). https://code.google.com/archive/p/program-ab/
- Pandorabots. AIML Guides & Playground. https://www.pandorabots.com/
- Wikipedia. Artificial Intelligence Markup Language y Loebner Prize. https://en.wikipedia.org/
- MyRobotLab. (2026). *pyrobotlab* (Versión master) [Software]. GitHub. https://github.com/MyRobotLab/pyrobotlab/tree/master

Anexos

Anexo A: Instrucciones de Ejecución del Proyecto

Para ejecutar el chatbot en cualquier entorno GNU/Linux

```
# 1. Descomprimir la entrega
unzip practica02-aiml.zip
cd practica02-aiml

# 2. Instalar la dependencia de AIML
pip install -r requirements.txt   # equivale a: pip install python-aiml

# 3. Ejecutar el asistente conversacional
python main.py

# 4. Ejecutar pruebas automáticas, para chequeo rapido
python probar.py -v
```

Anexo B: Estructura del Repositorio de Código

```
practica02-aiml/
├── main.py
├── normalizar.py
├── probar.py
└── requirements.txt
```

```
├── README.md / README.pdf
├── LICENSE
├── pruebas/
│   └── casos.py
└── archivos_aiml/
    ├── startup.aiml
    ├── negocio.aiml
    ├── saludos.aiml
    ├── despedidas.aiml
    ├── respGenerales.aiml
    ├── conectores.aiml
    ├── adverbios.aiml
    ├── horarios.aiml
    ├── ubicacion.aiml
    ├── pagos.aiml
    ├── productos.aiml
    ├── precios.aiml
    ├── promos.aiml
    ├── pedidos.aiml
    ├── envios.aiml
    ├── facturacion.aiml
    ├── devoluciones.aiml
    ├── soporte.aiml
    └── fallback.aiml
```